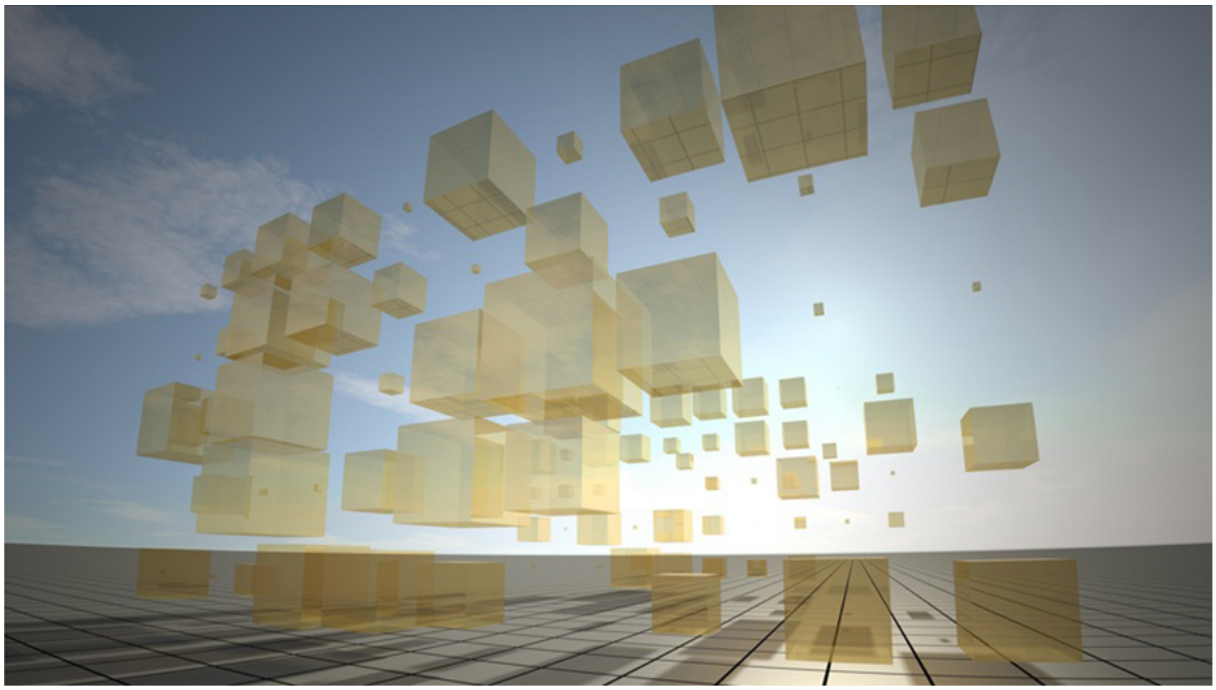
Hochleistungsdatenhaltungssysteme



Dr. Stephan Hofmann

Sommersemester 2024

TH Ingolstadt

Teil 5: Analyse unstrukturierter Daten – Teil V

INTERNAL

Überblick

Die nachfolgende Übung soll einen ersten Eindruck in den Umgang mit unstrukturierten Daten innerhalb einer SAP HANA geben.

Zielsetzung:

Nach dem Abschluss der Übung sollte die Implementierung einer fehler-toleranten Suche innerhalb der HANA-DB verstanden worden sein. Des Weiteren soll die grundsätzliche Vorgehensweise bei der integrierten Textanalyse verstanden werden.

INTERNAL

Inhalt

Umgang mit Textdaten 4
Exaktes Suchen in Datenbanken 4
Fehlertolerantes Suchen 5
Fehlertolerante Suche in SAP HANA: 6
Die SCORE() Function 6
Ein einfaches Beispiel: 6
Ein Anwendungsfall: Duplikatssuche 8
Aufgabe: 12
SAP HANA Text Analysis: 13
Kurze Einführung: 13
Anwendungsfall: 14
Voraussetzungen: 15
Auswertung der Textanalyse: 17
Weitere Analysen auf den erzeugten Ergebnissen: 18
Aufgabe: 19

Umgang mit Textdaten

- Fehlertolerantes Suchen in strukturierten Daten
- Speicherung von unstrukturierten Daten
- Kombination von strukturierten und unstrukturierten Daten
- Textanalyse - Extraktion von Bedeutungen aus Textdaten

Exaktes Suchen in Datenbanken

Daten über verschiedene Firmen sind strukturiert in einer Tabelle abgelegt. Bsp:

	ID	FIRMEN_NAME
1	1	AUDI
2	2	AUDI in Ingolstadt
3	3	AUDI AG
4	4	ADUI AG
5	5	BAUDI AG
6	6	BMW AG

Mit Hilfe der SQL-Anfrage können Einträge aus der Tabelle exakt bestimmt werden.

```
Select * from FIRMEN where FIRMEN_NAME = 'AUDI'
```

	ID	FIRMEN_NAME
1	1	AUDI

Da genau ein Eintrag den Firmen_Namen „AUDI“ trägt, wird genau ein Eintrag zurück gegeben.

Um flexiblere Abfragen zu ermöglichen, kann man Wildcards in die Abfragen integrieren. Innerhalb der Abfrage werden alle Einträge selektiert, für die es einen exakten Bestandteil innerhalb des Eintrages gibt. Hierfür kann das „like“ predicate in Kombination mit „%“ verwendet werden.

Bsp.:

```
Select * from FIRMEN where FIRMEN_NAME like '%AUDI%'
```

	ID	FIRMEN_NAME
1	1	AUDI
2	2	AUDI in Ingolstadt
3	3	AUDI AG
4	5	BAUDI AG

Bei der Verwendung von Wildcards, werden jedoch Groß- und Kleinschreibung berücksichtigt, sodass „audi“ ungleich „AUDI“ ist

SQL | Result

```
Select * from FIRMEN where FIRMEN_NAME like '%audi%'
```

ID	FIRMEN_NAME

Fehlertolerantes Suchen

Die hier betrachtete fehlertolerante Suche basiert auf einem approximate string matching Algorithmus. Damit adressiert man das Auffinden von Strings, die ein bestimmtes Muster approximieren. Es sind beispielsweise auch Schreibfehler tolerierbar oder das Suchen mit Wortbestandteilen.

Ein kleines Beispiel:

Wenn man beispielsweise "Dr Stephn Hofmann" bei Google eingibt (hier kommt approximate string matching zum Einsatz), kommt eine Liste an Treffern, welche den „richtigen“ Eintrag „Dr stephan hofmann“ beinhaltet. Die implementierte Suche ist somit eine fehlertolerante Suche.

Fehlertolerante Suche in SAP HANA:

In SAP HANA, kann man die “Fuzzy Suche” verwenden, indem das “CONTAINS predicate” mit zusätzlichen Optionen innerhalb der WHERE clause des SELECT Statements benutzt.

Hierbei kommt die folgende Syntax zum Einsatz:

Syntax:

SELECT * FROM <tablename>
WHERE CONTAINS (<column_name>, <search_string>, FUZZY (0.8))

Die entsprechende Suche mit der Option **FUZZY(x)** gibt alle Werte zurück, deren Scoring größer oder gleich 0.8 ist.

Die SCORE() Function

Der Suchalgorithmus berechnet für jeden String vergleich einen fuzzy score. Je höher dieser Score ist, desto ähnlicher sind die verglichenen Strings. Ein score von 1.0 bedeutet hierbei, dass beide String identisch sind. Ein score von 0.0 bedeutet das die beiden Strings keinerlei Gemeinsamkeiten haben.

Es ist möglich die ScoreFunktion innerhalb des SELECT Statements aufzurufen. Das macht beispielsweise Sinn, wenn die Ergebnisse anhand des Scorings sortiert ausgegeben werden sollen. Wenn innerhalb eines Statements mehrere Spalten betrachtet werden, wird der Gesamtscore als Durchschnitt aller Einzelscores ermittelt.

Ein einfaches Beispiel:

Wenn man beispielsweise nach dem Begriff 'AUDI' suchen. Ein Eintrag wie 'AUDI AG' bekommt einen hohen Score, denn der Term 'AUDI' existiert innerhalb des Textes. Ein Eintrag wie "BAUDI AG" bekommt einen niedrigeren Score, denn 'AUDI' ist nur ein Teil längeren Terms 'AUDI AG'.

Anlegen der Tabellen und Beladen mit Daten:

```
CREATE COLUMN TABLE FIRMEN(
               ID INTEGER PRIMARY KEY,
               FIRMEN_NAME SHORTTEXT(200) FUZZY SEARCH INDEX ON);
```

Table Name: FIRMEN | Schema: GBI_DEMO_DATA | Open Data

Columns | Indexes | Properties | Runtime Information

	Name	SQL Data Type	Column Store Data Type	Key	Not Null	Default	Comment
1	ID	INTEGER	INT	1	X	NULL	
2	FIRMEN_NAME	SHORTTEXT(200)	STRING			NULL	

Zusätzlich zur Tabelle „Firmen“ wird ein Full-Text-Index automatisch angelegt.
➔ FUZZY SEARCH INDEX ON

Table Name: FIRMEN | Schema: GBI_DEMO_DATA | Open Data

Columns | Indexes | Properties | Runtime Information

Name	Type	Indexed Columns
_SYS_TREE_CS_#167976_#0_#P0	INVERTED VALUE UNIQU	ID ASC
_SYS_FULLTEXT_167976_#0_#FIRMEN_NAME	FULLTEXT	FIRMEN_NAME

Filter Databases
- hdi_gbidemodata (D03)
  - Column Views
  - Cubes
  - DataStores
  - Functions
  - Graph Workspaces
  - Indexes
  - JSON Collections
  - Libraries
  - Procedures
  - Public Synonyms
  - Remote Subscriptions
  - Sequences
  - Synonyms
  - Table Types
  - Tables
  - Tasks
  - Triggers
  - Views
- pal-grantor-service (D03)

Search Indexes
- IDX_D000_BOOKING_DETAILS_1
- _SYS_FULLTEXT_167976_#0_#FIRMEN_NAME

Index Name: _SYS_FULLTEXT_167976_#0_#FIRMEN_NAME | Table Name: GBI_DEMO_DATA.FIRMEN | Type: Full-Text Index

Column Name: FIRMEN_NAME | Language | Mime Type Column
SQL Data Type: SHORTTEXT | Language Detection: EN | Mime Type
Column Store Data Type: STRING | Phrase Index Ratio: 0.2000000298023224 | Configuration

☑ Fuzzy Search Index ☐ Text Mining
☑ Search Only ☐ Text Analysis
☑ Fast Preprocess Token Separators: \/;,.:-_()[]<>!?*@+{}=“&#$~|

Synchronous means that an SQL insert (or update) statement will return after the data has been completely processed.
Asynchronous means that new data will be processed in the background by a queue, so that an SQL insert (or update) statement will return before the analysis has finished.

◉ Synchronous ○ Asynchronous
Every minutes
After documents

Dieser Index wird synchron beim Einfügen von Daten aktualisiert. Enthält einen Fuzzy Search Index Pattern und erlaubt ein Fast Preprocessing.

Über eine Reihe von Anweisungen werden Beispieldaten hinzugefügt:

```
INSERT INTO FIRMEN VALUES (1, 'AUDI');
INSERT INTO FIRMEN VALUES (2, 'AUDI in Ingolstadt');
INSERT INTO FIRMEN VALUES (3, 'AUDI AG');
INSERT INTO FIRMEN VALUES (4, 'ADUI AG');
INSERT INTO FIRMEN VALUES (5, 'BAUDI AG');
INSERT INTO FIRMEN VALUES (6, 'BMW AG');
```

Fehlertolerante Suche auf einer Spalte:

Unter Verwendung des Contains predicate innerhalb der folgenden Abfrage kann fehlertolerant gesucht werden.

```
SELECT SCORE() AS score, * FROM FIRMEN
       WHERE CONTAINS(FIRMEN_NAME,'AUDI',
```

```
                FUZZY(0.7))
        ORDER BY score DESC;
```

Die Ausgabe der Fuzzy Suche enthält 5 Einträge. Basierend auf dem Suchfaktor (Matching mit Score 0.7), werden alle Einträge betrachtet, die „AUDI“ ähnlich sind. Beispielsweise im Falle von "AUDI AG" oder "BAUDI AG" etc.

```
13 SELECT SCORE() AS score, * FROM FIRMEN
14         WHERE CONTAINS(FIRMEN_NAME,'AUDI',
15                 FUZZY(0.7))
16         ORDER BY score DESC;
17
```

Result × Messages × History

Rows (5) SQL

	SCORE	ID	FIRMEN_NAME
1	1	1	AUDI
2	0.95462965965271	3	AUDI AG
3	0.9274073839187622	2	AUDI in Ingolstadt
4	0.8001984357833862	5	BAUDI AG
5	0.5589285492897034	4	ADUI AG

Innerhalb des Fuzzy Suche wird beispielsweise Groß- und Kleinschreibung toleriert.

```
SELECT SCORE() AS score, * FROM FIRMEN
        WHERE CONTAINS(FIRMEN_NAME,'audi',
                FUZZY(0.7))
        ORDER BY score DESC;
```

```
17
18 SELECT SCORE() AS score, * FROM FIRMEN
19         WHERE CONTAINS(FIRMEN_NAME,'audi',
20                 FUZZY(0.7))
21         ORDER BY score DESC;
22
```

Result × Messages × History

Rows (5) SQL

	SCORE	ID	FIRMEN_NAME
1	1	1	AUDI
2	0.95462965965271	3	AUDI AG
3	0.9274073839187622	2	AUDI in Ingolstadt
4	0.8001984357833862	5	BAUDI AG
5	0.5589285492897034	4	ADUI AG

Ein Anwendungsfall: Duplikatssuche

Innerhalb einer Call-Center Anwendung werden Kundendaten strukturiert abgelegt. Um beispielsweise Kundendaten während eines Telefonats zu finden, ist eine fehlertolerante Suche notwendig, um beispielsweise bereits bestehende Einträge zu finden, die durch die telefonische Angabe nicht korrekt spezifiziert werden können.

Ein Beispiel: "Jimi Hendricks" kann sowohl als "Jimy Hendricks" oder als "Jimi Hendrix" etc. geschrieben werden. Auch die Adresse kann unterschiedlich

verstanden werden. Beispiel "Berliner Platz 43" oder "Berliner-Platz 43" oder nur "Berliner Platz".

Anlegen der Tabelle und importieren der Daten:

```
create column table "CUSTOMER"(
                "CUSTOMER_ID" VARCHAR (5) not null default '',
                "FIRST_NAME" VARCHAR (20) null default '',
                "LAST_NAME" VARCHAR (20) null default '',
                "STREET" VARCHAR (20) null default '',
                "CITY" VARCHAR (20) null default '',
                "COUNTRY" VARCHAR (20) null default '',
                "POSTAL_CODE" VARCHAR (20) null default '',
                primary key ("CUSTOMER_ID"));


insert into CUSTOMER values('00001','Jimi','Hendricks','Berliner Platz
43','Munchen','Germany','80805');
insert into CUSTOMER values('00002','Jimy','Hendricks','Berlinr Platz
43','Munchen','Germany','80805');
insert into CUSTOMER values('00003','Jimi','Hendrix','Berliner Plats
43','Munchen','Germany','80805');
insert into CUSTOMER
values('00004','Jimy','Feuer','Berliner','Munchen','Germany','80805');
insert into CUSTOMER values('00006','Sven','Ottlieb','Walserweg
21','Aachen','Germany','52066');
insert into CUSTOMER values('00007','Philip','Cramer','Maubelstr.
90','Brandenburg','Germany','14776');
insert into CUSTOMER values('00008','Renate','Messner','Magazinweg
7','Frankfurt','Germany','60528');
insert into CUSTOMER values('00009','Alexander','Feuer','Heerstr.
22','Leipzig','Germany','04179');
insert into CUSTOMER values('00010','Antonio','Moreno','Mataderos
2312','Mexico','Mexico','05023');
insert into CUSTOMER values('00011','Thomas','Hardy','120
Hanover','London','UK','WA1 1DP');
insert into CUSTOMER values('00012','Christina','Berglund','Berguvsvagen
8','Lulea','Sweden','S-958 22');
```

Abfrage ohne Fuzzy Search:

Innerhalb der Abfrage wird nach dem Kunden mit dem Namen "Jimi" gesucht.

SQL Query:

```
SELECT * FROM  CUSTOMER
                WHERE CONTAINS(FIRST_NAME, 'Jimi')
                ORDER BY "CUSTOMER_ID" DESC;
```

Die Ausgabe enthält damit nur die Einträge bei denen der Stringvergleich mit “Jimi” exakt ist.

```
SELECT * FROM  CUSTOMER
                WHERE CONTAINS(FIRST_NAME, 'Jimi')
                ORDER BY "CUSTOMER_ID" DESC;
```

Result × Messages × History

Rows (2) SQL

	CUSTOMER_ID	FIRST_NAME	LAST_NAME	STREET	CITY	COUNTRY	POSTAL_CODE
1	00003	Jimi	Hendrix	Berliner Plats 43	Munchen	Germany	80805
2	00001	Jimi	Hendricks	Berliner Platz 43	Munchen	Germany	80805

Abfrage mit Fuzzy Search:

Zunächst legen wir noch einen Full-Text-Index an. Dieser dient der Performanceoptimierung und kann auf relevanten Spalten angelegt werden.

```
CREATE FULLTEXT INDEX FT_IDX_NAME ON "GBI_DEMO_DATA"."CUSTOMER" ("FIRST_NAME") FUZZY SEARCH INDEX ON Sync;
```

Messages × History

```
Statement: CREATE FULLTEXT INDEX FT_IDX_NAME ON "GBI_DEMO_DATA"."CUSTOMER" ("FIRST_NAME") FUZZY SEARCH INDEX ...
Client elapsed time:    10.00 ms
Statement prepare time: 2.104 ms elapsed time, 1.672 ms CPU time
Statement execute time: 10.07 ms elapsed time, 8.428 ms CPU time
```

Abfrage mit Fehlertoleranz:

SQL Query:

```
SELECT SCORE() AS score, * FROM  CUSTOMER
                 WHERE
                CONTAINS(FIRST_NAME, 'Jimi', FUZZY(0.7))
                ORDER BY score DESC;
```

```
55 SELECT SCORE() AS score, * FROM  CUSTOMER
56                 WHERE
57                CONTAINS(FIRST_NAME, 'Jimi', FUZZY(0.7))
58                ORDER BY score DESC;
59
60
```

Result × Messages × History

Rows (4) SQL

	SCORE	CUSTOMER_ID	FIRST_NAME	LAST_NAME	STREET	CITY	COUNTRY	POSTAL_CODE
1	1	00001	Jimi	Hendricks	Berliner Platz 43	Munchen	Germany	80805
2	1	00003	Jimi	Hendrix	Berliner Plats 43	Munchen	Germany	80805
3	0.8833333253860474	00002	Jimy	Hendricks	Berlinr Platz 43	Munchen	Germany	80805
4	0.8833333253860474	00004	Jimy	Feuer	Berliner	Munchen	Germany	80805

Die Ausgabe enthält nun 4 Einträge. Basierend auf dem fuzzy search factor (FUZZY(0.7) innerhalb der Abfrage), werden alle ähnlichen Einträge zu „Jimi“ ermittelt.

Abfrage mit Fehlertoleranz auf mehreren Spalten:

Die Suche kann auch auf mehrere Spalten angewendet werden. Beispielsweise auf First Name und Last Name.

SQL Query:

```
SELECT SCORE() AS score, * FROM  CUSTOMER
                 WHERE CONTAINS(FIRST_NAME, 'Jimi', FUZZY(0.7))
                       and CONTAINS(LAST_NAME, 'Hendricks', FUZZY(0.7))
                 ORDER BY score DESC;
```

Die Ausgabe enthält nun 3 Einträge . Basierend dem fuzzy search factor (FUZZY(0.7) innerhalb der Abfrage), wird der Durschnitt der scores auf den Spalten ermittelt und damit ähnliche Einträge für die Kombination “Jimi Hendricks”

ermittelt. In diesem Fall “Jimi Hendricks” , "Jimy Hendricks" und "Jimi Hendrix".

```
60 SELECT SCORE() AS score, * FROM  CUSTOMER
61                WHERE CONTAINS(FIRST_NAME, 'Jimi', FUZZY(0.7))
62                      and CONTAINS(LAST_NAME, 'Hendricks', FUZZY(0.7))
63                ORDER BY score DESC;
64
65
66
```

Result × Messages × History

Rows (3) SQL

	SCORE	CUSTOMER_ID	FIRST_NAME	LAST_NAME	STREET	CITY	COUNTRY	POSTAL_CODE
1	1	00001	Jimi	Hendricks	Berliner Platz 43	Munchen	Germany	80805
2	0.9175041913986206	00002	Jimy	Hendricks	Berlinr Platz 43	Munchen	Germany	80805
3	0.8256991505622864	00003	Jimi	Hendrix	Berliner Plats 43	Munchen	Germany	80805

Aufgabe:

In den Tabellen „D000_Customer“ und „D000_Supplier“ sind beispielhaft Daten über die Kunden und Lieferanten einer exemplarischen Firma abgelegt.

Übernehmen Sie die Daten der beiden Tabellen in ihre eigenen Tabellen. Legen Sie die entsprechenden Tabellen: Dxxx_Customer und Dxxx_Supplier an und kopieren Sie die existierenden Daten aus den besehenden Tabellen D000_Customer/Supplier.

Filter Databases
- hdi_gbidemodata (D03)
  - Column Views
  - Cubes
  - DataStores
  - Functions
  - Graph Workspaces
  - Indexes
  - JSON Collections
  - Libraries
  - Procedures
  - Public Synonyms
  - Remote Subscriptions
  - Sequences
  - Synonyms
  - Table Types
  - Tables
  - Tasks
  - Triggers
  - Views
- pal-grantor-service (D03)

D000_Cus

D000_CUSTOMER

Table Name: D000_CUSTOMER | Schema: GBI_DEMO_DATA | Open Da

Columns Indexes Properties Runtime Information

	Name	SQL Data Type	Column Store Data Type	Key	Not Null	Default	Comment
1	KUNNR	NVARCHAR(10)	STRING	1	X		
2	LAND1	NVARCHAR(3)	STRING				
3	NAME1	NVARCHAR(35)	STRING				
4	NAME2	NVARCHAR(35)	STRING				
5	ORT01	NVARCHAR(35)	STRING				
6	PSTLZ	NVARCHAR(10)	STRING				
7	REGIO	NVARCHAR(3)	STRING				
8	SORTL	NVARCHAR(10)	STRING				
9	STRAS	NVARCHAR(35)	STRING				
10	TELF1	NVARCHAR(16)	STRING				
11	ANRED	NVARCHAR(15)	STRING				

- hdi_gbidemodata (D03)
  - Column Views
  - Cubes
  - DataStores
  - Functions
  - Graph Workspaces
  - Indexes
  - JSON Collections
  - Libraries
  - Procedures
  - Public Synonyms
  - Remote Subscriptions
  - Sequences
  - Synonyms
  - Table Types
  - Tables
  - Tasks
  - Triggers
  - Views
- pal-grantor-service (D03)

D000_SU

D000_SUPPLIER

D000_SUPPLIER | GBI_DEMO_DATA

Columns Indexes Properties Runtime Information

	Name	SQL Data Type	Column Store Data Type	Key	Not Null	Default
1	LIFNR	NVARCHAR(10)	STRING	1	X	
2	LAND1	NVARCHAR(3)	STRING			
3	NAME1	NVARCHAR(35)	STRING			
4	NAME2	NVARCHAR(35)	STRING			
5	NAME3	NVARCHAR(35)	STRING			
6	NAME4	NVARCHAR(35)	STRING			
7	ORT01	NVARCHAR(35)	STRING			
8	STRAS	NVARCHAR(35)	STRING			
9	ANRED	NVARCHAR(15)	STRING			

1. Prüfen Sie die Struktur der beiden Tabellen. In welchen Spalten befinden sich Informationen über den Namen bzw. die Bezeichnung des Kunden oder Lieferanten.
2. Erstellen sie für beide Tabellen (Dxxx) einen Fulltextindex. Nutzen Sie die folgende Konvention: FT_IDX_Dxxx_<Attribut>_C und FT_IDX_Dxxx_<Attribut>_S
3. Finden Sie alle Firmen heraus, die etwas herstellen. Nutzen Sie hierfür einmal den Begriff „Manufactoring“ und ein weiteres Mal „Manufacturing“ mit einem Fuzzy-Search Faktor von 0.5. Wie verändert sich die Ergebnismenge und das Scoring der Top-Treffer?
4. Finden Sie alle Firmen heraus, die etwas liefern. Nutzen Sie hierfür einmal den Begriff „supplies“ “ mit einem Fuzzy-Search Faktor von 0.3. Wie kann man über das Ergebnis sagen.

SAP HANA Text Analysis:

Kurze Einführung:

Mit Hilfe der integrierten Regeln wird versucht aus dem Text wesentliche Bestandteile zu extrahieren und zu bewerten. Aus Textelementen werden Sentiments abgeleitet.

Bsp:

I am [happy].	→ Weak Positive
I am [not happy].	→ Weak Negative
I am [tremendously happy].	→ Strong Positive
I am [not tremendously happy].	→ Weak Negative
I am [happy with my purchase].	→ Weak Positive
I am [not happy with my purchase].	→ Weak Negative
I am [tremendously happy with my purchase].	→ Strong Positive
I am [not tremendously happy with my purchase].	→ Weak Negative

Kombination von Regeln ermittelt die Ausgabe:

	Two Rule Sets	Output
I am <not [happy> with my purchase].	→ “happy” + “my purchase” → “happy” + “not”	→ WeakNegative
I am <tremendously [happy> with my purchase].	→ “happy” + “my purchase” → “happy” + “tremendously”	→ StrongPositive
I am <not tremendously [happy> with my purchase].	→ “happy” + “my purchase” → “happy” + “tremendously” + “not”	→ WeakNegative

Zur Analyse von Texten sind verschiedene Methoden in Kombination mit verschiedenen Sprachen verfügbar:

Available Text Analysis Configuration Options

Language	LINGANALYSIS_BASIC LINGANALYSIS_STEMS	LINGANALYSIS_FULL	EXTRACTION_CORE	EXTRACTION_CORE_VOICEOFCUSTOMER	EXTRACTION_CORE_PUBLIC_SECTOR	EXTRACTION_CORE_ENTERPRISE
Arabic	■	■	■			
Catalan	■	■				
Chinese (Simplified)	■	■	■	■ IMPROVED		
Chinese (Traditional)	■	■	■ NEW	■ NEW		
Croatian	■	■				
Czech	■	■				
Danish	■	■				
Dutch	■	■	■	■ NEW (Emoticons & Profanity only)		
English	■	■	■	■ IMPROVED	■ NEW	■ NEW
Farsi	■	■	■			
French	■	■	■	■ IMPROVED		
German	■	■	■	■ IMPROVED		
Greek	■					
Hebrew	■	■ NEW				
Hungarian	■					
Indonesian	■ NEW	■ NEW				
Italian	■	■	■	■ NEW		
Japanese	■	■	■			
Korean	■	■	■			
Norwegian (Bokmal)	■	■				
Norwegian (Nynorsk)	■	■				
Polish	■					
Portuguese	■	■	■	■ NEW		
Romanian	■					
Russian	■	■	■	■ NEW		
Serbian	■	■ NEW (Cyrillic support)				
Slovak	■	■				
Slovenian	■	■				
Spanish	■	■	■	■ IMPROVED		
Swedish	■	■				
Thai	■	■ NEW				
Turkish	■	■ NEW				

Quelle: SAP 2014

Eine Analysevariante ist: 'EXTRACTION_CORE_VOICEOFCUSTOMER'

Diese extrahiert 7 verschiedene Sentiment Level in Verbindung mit verschiedenen Sprachen.

Anwendungsfall:

Wenn man darüber nachdenkt sich ein neues Smartphone zu kaufen, kann die Analyse von entsprechenden Produktberichten im Internet durchaus hilfreich sein. Hierbei sind sowohl positive als auch negative Einträge in Blogs, offizielle Berichte, Kommunikation über Social Media etc. interessant.
Ein kleines Beispiel:

Strong demand of Apple's iPhone 5 series driving an "anti-fragmentation" of iOS

Just three weeks after Apple launched the new iPhone 5c and 5s, total iPhone Web traffic share attributed to iPhone 5 or newer models is up more than four percentage points to 40.6 percent, highlighting a rapid transition to modern hardware that's the exact opposite of the fragmentation occurring on Android.

Data from Web ad network Chitika tracking the source of mobile web requests in North America indicates what the company called "a "remarkable achievement" and represents a "significant impact on the iPhone ecosystem," with "obvious implications for mobile application and Web developers in terms of compatibility and functionality issues, but also for the technology industry as a whole."

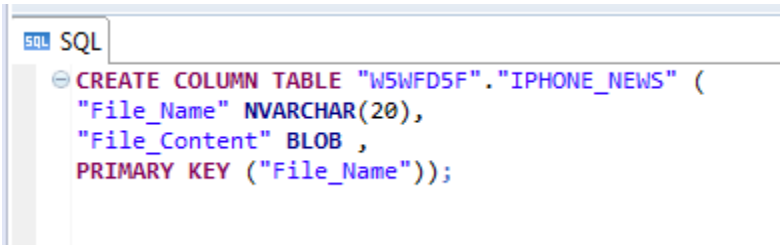
While Apple is already known to have a growing percentage of smartphone sales (via Kantar) andsubscribers (via comScore) in North America, the premium route buyers are taking to arrive on iOS is also newsworthy.

Voraussetzungen:

Zum Ablegen unstrukturierter Daten wird eine Columnar-Tabelle benötigt. Die Ablage der Daten erfolgt innerhalb eines BLOB-Objektes.

Folgende Anweisung erstellt eine entsprechende Struktur:

```
CREATE COLUMN TABLE <SCHEMA_NAME>."IPHONE_NEWS" (
"File_Name" NVARCHAR(20),
"File_Content" BLOB ,
PRIMARY KEY ("File_Name"));
```



Die Roh-Daten können dann per Import in die Datenbank geladen werden.

Table Name: IPHONE_NEWS | Schema: W5WFD5F | Type: Column Store

Columns | Indexes | Further Properties | Runtime Information

General

Total Memory Consumption (KB):	319	Memory Consumption in Main Storage (KB):	4
Number of Entries:	1	Memory Consumption in Delta Storage (KB):	315
Size on Disk (KB):	1.036	Estimated Maximum Memory Consumption (KB):	-

Details for Table

Parts | Columns

Host:Port/Partition/Sub-Parti...	Part ID	Range	Total Size (KB)	Main Size (KB)	Delta Size (KB)	Estimated Maximum Size (...	Record Cou...	Creation Time	Last Delta Merge
isbc002:30103			319	4	315	N/A	1	19.06.2015 09:10:43	

Implementierung der Text Analysis in SAP HANA:

Die Textanalyse innerhalb der Datenbank ist relativ einfach. Basierend auf den Rohdaten ist ein Full-Text-Index anzulegen, der nach einem integrierten Verfahren die interessanten Elemente aus den Rohdaten extrahiert und bewertet.

Create FullText Index "PDF_FTI" On <SCHEMA_NAME>."IPHONE_NEWS"("File_Content")

TEXT ANALYSIS ON

LANGUAGE DETECTION ('en')

CONFIGURATION 'EXTRACTION_CORE_VOICEOFCUSTOMER';

AHA (W5WFD5F) isbc002p.in.audi.vwg 01

SQL

```
Create FullText Index "PDF_FTI" On "W5WFD5F"."IPHONE_NEWS"("File_Content")
TEXT ANALYSIS ON
CONFIGURATION 'EXTRACTION_CORE_VOICEOFCUSTOMER';
```

```
Statement 'CREATE COLUMN TABLE "W5WFD5F"."IPHONE_NEWS" ( "File_Name" NVARCHAR(20), "File_Content" BLOB , ...'
successfully executed in 6 ms 211 µs  (server processing time: 4 ms 977 µs) - Rows Affected: 0

Statement 'Create FullText Index "PDF_FTI" On "W5WFD5F"."IPHONE_NEWS"("File_Content") TEXT ANALYSIS ON ...'
successfully executed in 14 ms 445 µs  (server processing time: 13 ms 402 µs) - Rows Affected: 0
```

Mit Hilfe der Anweisung wird der full text index "PDF_FTI" auf der BLOB Spalte "File_Content" der Tabelle "IPHONE_NEWS"erzeugt.

Table Name: IPHONE_NEWS | Schema: W5WFD5F | Type: Column Store

Columns | Indexes | Further Properties | Runtime Information

	Name	Type	Indexed Columns
1	PDF_FTI	Full-Text Index	"File_Content"

Column Name	SQL Data Type	Column Store Dat...	Sort	Language	Language Detection
File_Content	BLOB	LOB	ASC	Not defined	'EN'

Configuration	Text Mining	Text Analysis
EXTRACTION_CORE_VOICEOFCUSTOMER	☐	☑

Hierdurch wird eine Tabelle erzeugt, die den Inhalt des Indexes speichert.

Innerhalb dieser Tabelle sind die Ergebnisse der Textanalyse abgelegt.

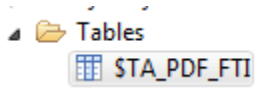


Table Name: $TA_PDF_FTI Schema: W5WFD5F Type: Column Store

Columns | Indexes | Further Properties | Runtime Information

	Name	SQL Data Type	Di...	Column Store Data Type	Key	Not Null	Default	Comment
1	File_Name	NVARCHAR	20	STRING	X(1)	X		
2	TA_RULE	NVARCHAR	200	STRING	X(2)	X		
3	TA_COUNTER	BIGINT		FIXED	X(3)	X		
4	TA_TOKEN	NVARCHAR	5000	STRING				
5	TA_LANGUAGE	NVARCHAR	2	STRING				
6	TA_TYPE	NVARCHAR	100	STRING				
7	TA_NORMALIZED	NVARCHAR	5000	STRING				
8	TA_STEM	NVARCHAR	5000	STRING				
9	TA_PARAGRAPH	INTEGER		INT				
10	TA_SENTENCE	INTEGER		INT				
11	TA_CREATED_AT	TIMESTAMP		LONGDATE				
12	TA_OFFSET	BIGINT		FIXED				
13	TA_PARENT	BIGINT		FIXED				

Nach einem Refresh auf dem Schema, wird die neue Tabelle angezeigt.

Auswertung der Textanalyse:

```
SELECT TOP 1000 * FROM "W5WFD5F"."$TA_PDF_FTI"
```

	File_Name	TA_RULE	TA_COUNTER	TA_TOKEN	TA_LANGUAGE	TA_TYPE
1	NEWS.pdf	Entity Extraction	1	Strong demand of Apple	en	Sentiment
2	NEWS.pdf	Entity Extraction	2	Strong	en	WeakPositiveSentiment
3	NEWS.pdf	Entity Extraction	3	demand of Apple	en	Topic
4	NEWS.pdf	Entity Extraction	4	Apple	en	ORGANIZATION/COMMERCIAL
5	NEWS.pdf	Entity Extraction	5	iPhone 5	en	PRODUCT
6	NEWS.pdf	Entity Extraction	6	iOS	en	PRODUCT

Weitere Analysen auf den erzeugten Ergebnissen:

2 Spalten der Tabelle $TA_PDF_FTI sind interessant:

TA_TOKEN (Text Analysis Token) Diese Spalte enthält die extrahierten Entitäten oder Elemente der Text-Analyse (beispielsweise eine Person, ein Ort, ein Thema, eine Organisation, oder eine Stimmung (Sentiment)).

In unserem Beispiel: Strong, Apple, iPhone 5, iOS.

TA_TYPE (Text Analysis Typ)

Diese Spalte enthält die Kategorie der identifizierten Objekte.

Die detaillierte Bewertung des Reviews, kann mit weiteren Abfragen erfolgen.

Hier ein Beispiel: Um herauszufinden, wie die Kunden über das iPhone5 denken bzw. welche Emotionen über das iPhone 5 innerhalb des Dokumentes zu finden sind, kann man die folgende Abfrage auf der Tabelle: $TA_PDF_FTI starten.

```
SELECT "TA_TYPE", ROUND("SENTIMENT_VALUE"/ "TOTAL_SENTIMENT_VALUE" * 100,2) AS
"SENTIMENT_VALUE_PERCENTAGE"
FROM
(
SELECT "TA_TYPE", SUM("TA_COUNTER") AS "SENTIMENT_VALUE"
FROM "W5WFD5F"."$TA_PDF_FTI"
where TA_TYPE
in('WeakPositiveSentiment','StrongPositiveSentiment','NeutralSentiment',
'WeakNegativeSentiment','StrongNegativeSentiment','MajorProblem','MinorProblem')
GROUP BY "TA_TYPE"
) AS TABLE1,
(
SELECT SUM("TA_COUNTER") AS "TOTAL_SENTIMENT_VALUE"
FROM "W5WFD5F"."$TA_PDF_FTI"
where TA_TYPE
in('WeakPositiveSentiment','StrongPositiveSentiment','NeutralSentiment',
'WeakNegativeSentiment','StrongNegativeSentiment','MajorProblem','MinorProblem')
) AS TABLE2
```

Die Ausgabe der Abfrage sieht beispielhaft so aus.

	TA_TYPE	SENTIMENT_VALAUE_PERCENTAGE
1	WeakPositiveSentiment	56.51
2	StrongPositiveSentiment	9.74
3	MinorProblem	14.82
4	StrongNegativeSentim...	1.75
5	WeakNegativeSentiment	8.98
6	MajorProblem	3.49
7	NeutralSentiment	4.7

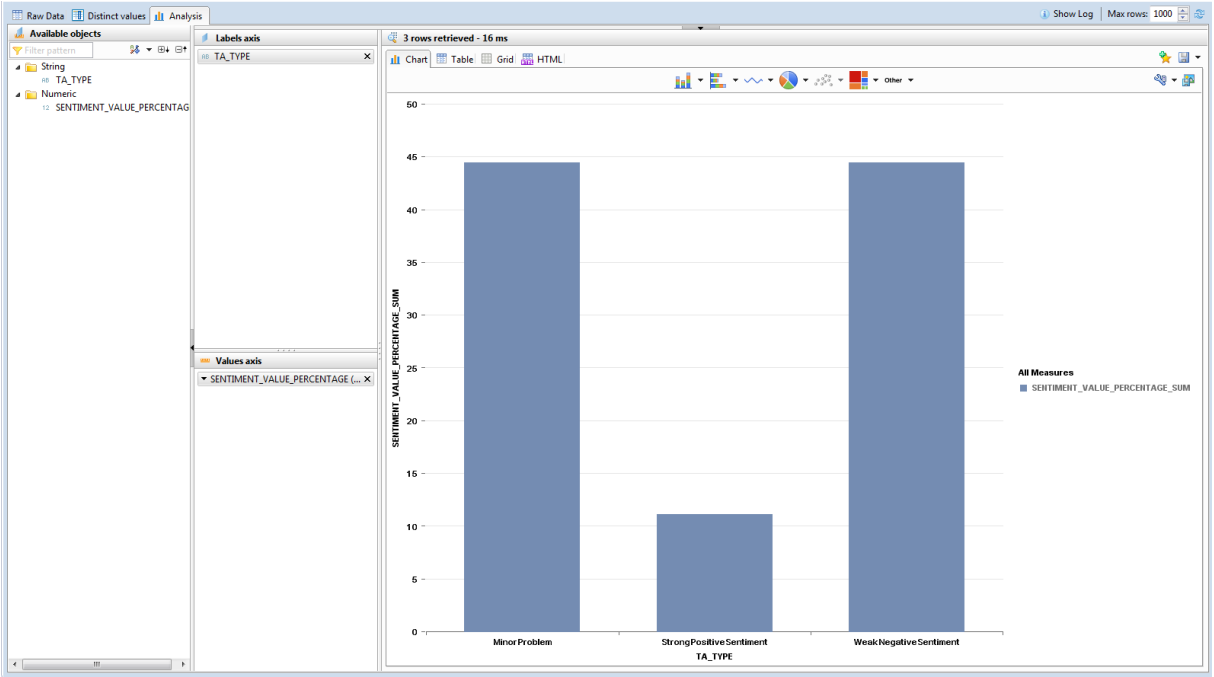
Damit zeigt sich, dass grundsätzlich ein positives Feedback über das iPhone gefunden wurde. --> Aber wer kauft sich jetzt noch ein iPhone 5 :-)

Aufgabe:

Ein gemeinsamer Anwendungsfall wird erarbeitet und umgesetzt. Er beinhaltet die folgenden Schritte:

→ Suchen Sie Artikel im WWW zum Thema „Europawahl“ in Deutsch und analysieren Sie die Artikel.

- Definition geeigneter Tabellen, um dynamisch News abzuspeichern.
- Die Definition und Anlage des Textanalyse-Indizes zur Analyse deutscher Texte mit geeigneter Datensynchronisierung.
- Die Definition eines geeigneten Views zur Analyse und grafischen Darstellung der Ergebnisse.
- Das Befüllen der Tabelle und Auswerten des Ergebnisses in Echtzeit.


Raw Data
Distinct values
Analysis
Show Log
Max rows: 1000
Available objects
Filter pattern
String
TA_TYPE
Numeric
SENTIMENT_VALUE_PERCENTAG
Labels axis
TA_TYPE
Values axis
SENTIMENT_VALUE_PERCENTAGE (...
3 rows retrieved - 16 ms
Chart
Table
Grid
HTML
Other
SENTIMENT_VALUE_PERCENTAGE_SUM
50
45
40
35
30
25
20
15
10
5
0
Minor Problem
Strong Positive Sentiment
Weak Negative Sentiment
TA_TYPE
All Measures
SENTIMENT_VALUE_PERCENTAGE_SUM